

CS4423: Networks Summer 20/21

Cian Mac Liatháin

11476048

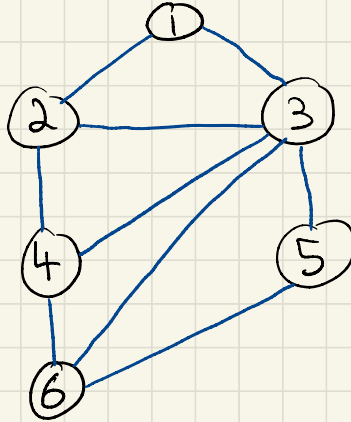
Extra time 10 mph



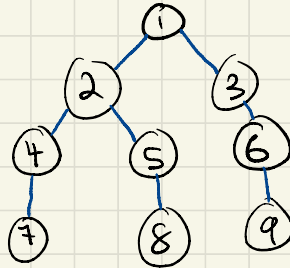
i) Order = $|V| = 6$
Size = $|E| = 9$

This is a graph $G = (V, E)$ with
 V the set of vertices and E
the set of edges, every edge
connects two vertices

This is a graph and not
a tree due to the presence
of cycles e.g. $(1, 2, 3, 1)$



ii) This is a tree because all
nodes are connected and
there are no cycles



(b) i)

Breadth-first search is an algorithm that can be used to travel from a starting node (input) and every other node in its component, generating a list of nodes as it traverses the component as its output. The steps taken by the algorithm involve identifying the neighbours of the root node and adding them to a First In First Out queue data structure which allows for values to be pushed/added to the end of the queue or popped/removed from the top of the queue. The algorithm continues pushing the neighbours of these neighbours in turn, until all nodes are visited and output in an order that grows further away from the starting node.

In a rooted tree with root x

$Q \leftarrow (x)$

repeat while $Q \neq \emptyset$:

$y \leftarrow Q.\text{pop}()$

visit(y)

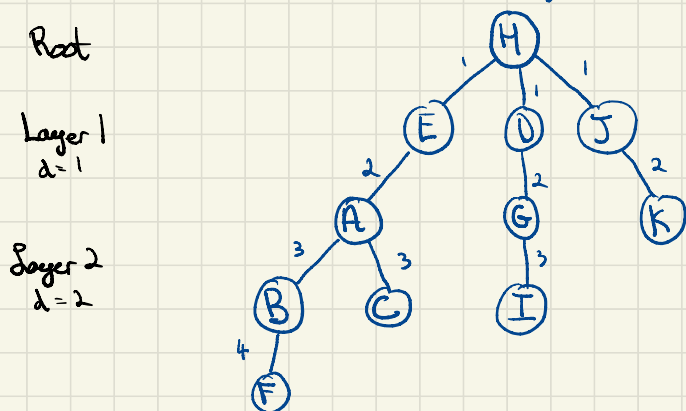
$Q.\text{push}(y \text{ children})$

output(y , queue)

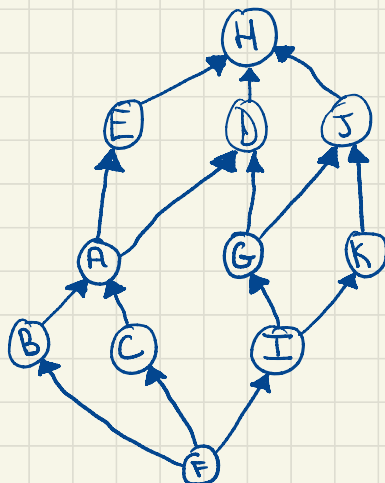
In an undirected graph with cycles, the algorithm builds on the simple implementation above as it must account for different paths to the same nodes. When neighbours are first identified they are marked as seen and attributed a distance from the starting node. The first layer of neighbours is distance $d=1$ from the starting node, and the algorithm repeats on each of these neighbours to find the next layer $d+1=2$ from the starting node and so forth.

This process, and variants of it can be used to ^{efficiently} generate spanning trees, measure shortest distances between nodes, and identify shortest paths between nodes, among other things.

(ii) Root



(iii) BFS was used to draw a shortest paths 'tree', with arrows pointing to possible predecessors on a shortest path from H to x.



Shortest paths from H to F:

(H, E, A, B, F)

(H, E, A, C, F)

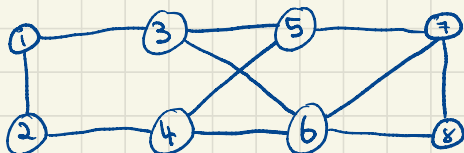
(H, D, A, C, F)

(H, D, G, I, F)

(H, J, K, I, F)

(H, J, G, I, F)

2.1,



(ii) $C_1^0 = 2$

$C_2^0 = 2$

$C_3^0 = 3$

$C_4^0 = 3$

$C_5^0 = 3$

$C_6^0 = 4$

$C_7^0 = 3$

$C_8^0 = 2$

(iii) 4 nodes have odd degree, every edge connects two nodes in the graph: $\sum_i k_i = 2m$

Thus, Euler's handshake lemma states that every simple graph has an even number of nodes of odd degree

(iv) $C_i = \begin{cases} \binom{k_i}{2} m(G_i), & k_i \geq 2 \\ 0 & \text{else} \end{cases}$ where $m(G_i)$ = number of edges of the subgraph induced on the neighbors of i in G

$C_1 = \binom{2}{2}^0 = 0$

$C_2 = \binom{2}{2}^0 = 0$

$C_3 = \binom{3}{2}^1 = 0$

$C_4 = \binom{3}{2}^1 = 0$

$C_5 = \binom{3}{2}^1 = 0$

$C_6 = \binom{4}{2}^1 = \frac{1}{6}$

$C_7 = \binom{3}{2}^1 = \frac{1}{3}$

$C_8 = \binom{2}{2}^1 = 1$

(v) $C = \langle C \rangle = \frac{1}{n} \sum_{i=1}^n C_i$
 $= \frac{\frac{1}{6} + \frac{1}{3} + 1}{8} = \frac{9/6}{8} = \frac{3}{16}$

$T = \frac{3n_A}{n \cdot n}$
 $= \frac{3}{21} = \frac{1}{7}$

$n_A = 1$

$n_A = \sum_{i=1}^n \binom{k_i}{2} = \binom{2}{2} + \binom{2}{2} + \binom{3}{2} + \binom{3}{2} + \binom{3}{2} + \binom{4}{2} + \binom{3}{2} + \binom{2}{2}$
 $= 1 + 1 + 3 + 3 + 3 + 6 + 3 + 1$
 $= 21$

- (ii) The graph is not bipartite as there exists a cycle of length 3:
The disjoint union of a bipartite graph means that all bipartite graphs have even cycles only.

(6, 8, 7, 6)

2 (b)

- (i) The tree is minimally connected
The tree is maximally connected
The tree is acyclic and $|E| = |V| - 1$

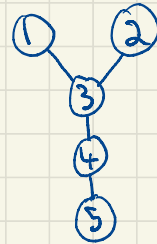
- (ii) The Prüfer code of a tree on n vertices is a unique sequence of $n-2$ vertices $\in X$

The Prüfer code of a tree T can be computed by:
identifying the smallest node of degree 1, x
add its unique neighbour to the sequence, y
Remove x and the edge $x-y$ from T
Repeat until T has 2 nodes left.

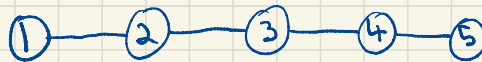
The degree of node x is 1 plus the number of entries x in Prüfer code of T

- (iii) $x=1: 3$
 $x=2: 3$
 $x=3: 4$

Prüfer Code: 3, 3, 4



- (iv) $\deg(1) = 1$
 $\deg(2) = 2$
 $\deg(3) = 2$
 $\deg(4) = 2$
 $\deg(5) = 1$



3 (a)

- (i) Erdős-Rényi model $g(n, m)$ graphs take the inputs of n nodes and m edges. This means that there is a maximum $\binom{n}{2}$ opportunities in each (simple) graph to add an edge, and this model encompasses the set of all possible graphs g with m edges whose nodes are selected randomly.

A $g(n, m)$ graph can be generated by initialising the graph with n nodes and no edges, and an empty counter^a measuring how many nodes have been visited, and another^c counting how many edges have been added.

Cycling through each node, an edge may be added if a random number $(0 \leq i < 1) \left(\binom{n}{2} - a \right) < m - c$. This distributes the edges uniformly at random. When the target m is reached, $m - c$ will be 0 and no more edges will be added.

- (ii) A $g(n, p)$ graph can be generated by initialising the graph with n nodes and no edges. Cycling through every node, an edge may be added between two nodes if a randomly generated number $0 \leq i < 1$ is less than the probability p .

- (iii) There are $\binom{n}{2}$ potential edges

The probability for to generate a graph with m edges is p^m edge : p^m

The probability of not generating other edges is $(1-p)^{\binom{n}{2}-m}$

Probability of a graph g being generated with m edges is $p^m (1-p)^{\binom{n}{2}-m}$

Probability of choosing one of these graphs from the $g(n, p)$ model is $\binom{\binom{n}{2}}{m} p^m (1-p)^{\binom{n}{2}-m}$

- (iv) $p = 1/6$

$$p_k = \binom{n-1}{k} p^k (1-p)^{n-1-k}$$

$$p_{10} = \binom{99}{10} \left(\frac{1}{6}\right)^{10} \left(1 - \frac{1}{6}\right)^{89}$$

$$= 0.023116$$

- (b) An equivalence relation is reflexive, symmetric and transitive.

- (i) $x \equiv y$: $x \rightsquigarrow y$ and $y \rightsquigarrow x$ where $x \rightsquigarrow y$ means there is a path from x to y .
This is reflexive, symmetric and transitive

- (ii) $x \sim y$: there is a path from x to y .

(15)

The other parts of the graph can then be divided into 4 other sets of components:

1. IN: These are upstream of the SCC, the set of components C that have a path into the giant SCC but no path back: $C < SCC$
2. OUT: These are downstream of the giant SCC, the set of components that can be reached from the giant SCC and have no path back: $C > SCC$
3. Tendrils are components that can be reached only by those in IN, with no path back, $C \not\rightarrow IN$ or those that can only reach the OUT set of components with no path back, $C \not\leftarrow OUT$.
Tubes are components that can be reached from the IN set and can reach the OUT set, without having a path to the giant SCC: $C > IN \& C < OUT \& C \not\leftarrow SCC$
4. Disconnected components have no path to the SCC, IN or OUT sets.

4 (a)

(i) The Configuration Model can be used by generating a list of stubs (half-edges) from the degree sequence where each node i has d_i stubs. By randomly matching pairs of stubs to reform edges, a random graph can be formed.

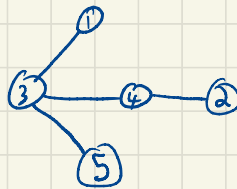
(ii) $X = \{1, 2, 3, 4, 5\}$

$$d_1 = 1, d_2 = 1, d_3 = 3, d_4 = 2, d_5 = 1$$

stubs = (1, 2, 3, 3, 3, 4, 4, 5)

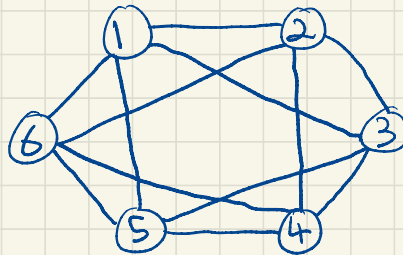
(3, 1, 2, 3, 4, 3, 4, 5)

$$\begin{pmatrix} 3 & 1 & 2 & 3 \\ 4 & 3 & 4 & 5 \end{pmatrix}$$



Edges
3-4
1-3
2-4
3-5

(b) (i) $n = 6, d = 2$



(ii) An (n, d, p) -WS graph begins as an (n, d) -circular graph, and cycling through each node $i \in X$, the nodes' edges connected to nodes within $1 \leq d < n/2$ distance are rewired to another node $j \neq i$ and where the edge doesn't already exist, with probability p .

(iii) $n = 6, d = 2, p = 0.25$

6 nodes

each node looks at 4 edges

$$(6)(4)(0.25) = 6 \text{ edges}$$

(iv) A random WS model graph exhibits small-world behaviour: high clustering and low characteristic path length, which aren't both found in random $\lg(n, m)$, $\lg(n, p)$ or (n, d) -circular graphs.

